Unit5:Machine Learning for DataScience

Machine Learning (ML)

Machine Learning is a branch of AI where systems learn patterns from data and improve performance without being explicitly programmed.

In Data Science, ML is used to predict outcomes, classify data, and find patterns.

Types of Machine Learning

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning

Supervised Learning

- Uses labeled data
- Input $\rightarrow$ Output mapping is known

Examples:

- Spam detection
- House price prediction

Algorithms:

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machine (SVM)

Unsupervised Learning

- Uses unlabeled data
- Finds hidden patterns

Examples:

- Customer segmentation
- Market basket analysis

Algorithms:

- K-Means Clustering
- Hierarchical Clustering
- PCA (Dimensionality Reduction)

Reinforcement Learning

- Learns by trial and error
- Uses rewards and penalties

Examples:

- Robotics
- Game playing (Chess, AlphaGo)

Machine Learning Workflow

- Data Collection
- Data Cleaning (missing values, noise)
- Data Exploration (EDA)
- Feature Engineering
- Model Selection
- Training the Model
- Testing & Evaluation
- Deployment

Linear Regression – Introduction

Linear Regression is a supervised learning algorithm used to predict a continuous value based on input features.

It finds the best-fit line between input (X) and output (Y).

Simple Linear Regression Formula

$$y = mx + c$$

- y = predicted output
- x = input variable
- m = slope (weight)
- c = intercept

Meaning of Parameters

- **Slope (m)**: Rate of change of output with respect to input
- **Intercept (c)**: Value of y when $x = 0$

It defines the position and tilt of the best-fit line.

Multiple Linear Regression

$$y = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_n x_n$$

- Multiple input features
- Each feature has its own weight
- b_0 is the intercept

Error Function (MSE)

$$MSE = \frac{1}{n}\sum(y - \hat{y})^2$$

- Measures prediction error
- Goal: minimize MSE
- $\hat{y}$ = predicted value

Working of Linear Regression

- Fits a best line through data points
- Minimizes error between actual and predicted values
- Uses Least Squares method

Advantages & Limitations

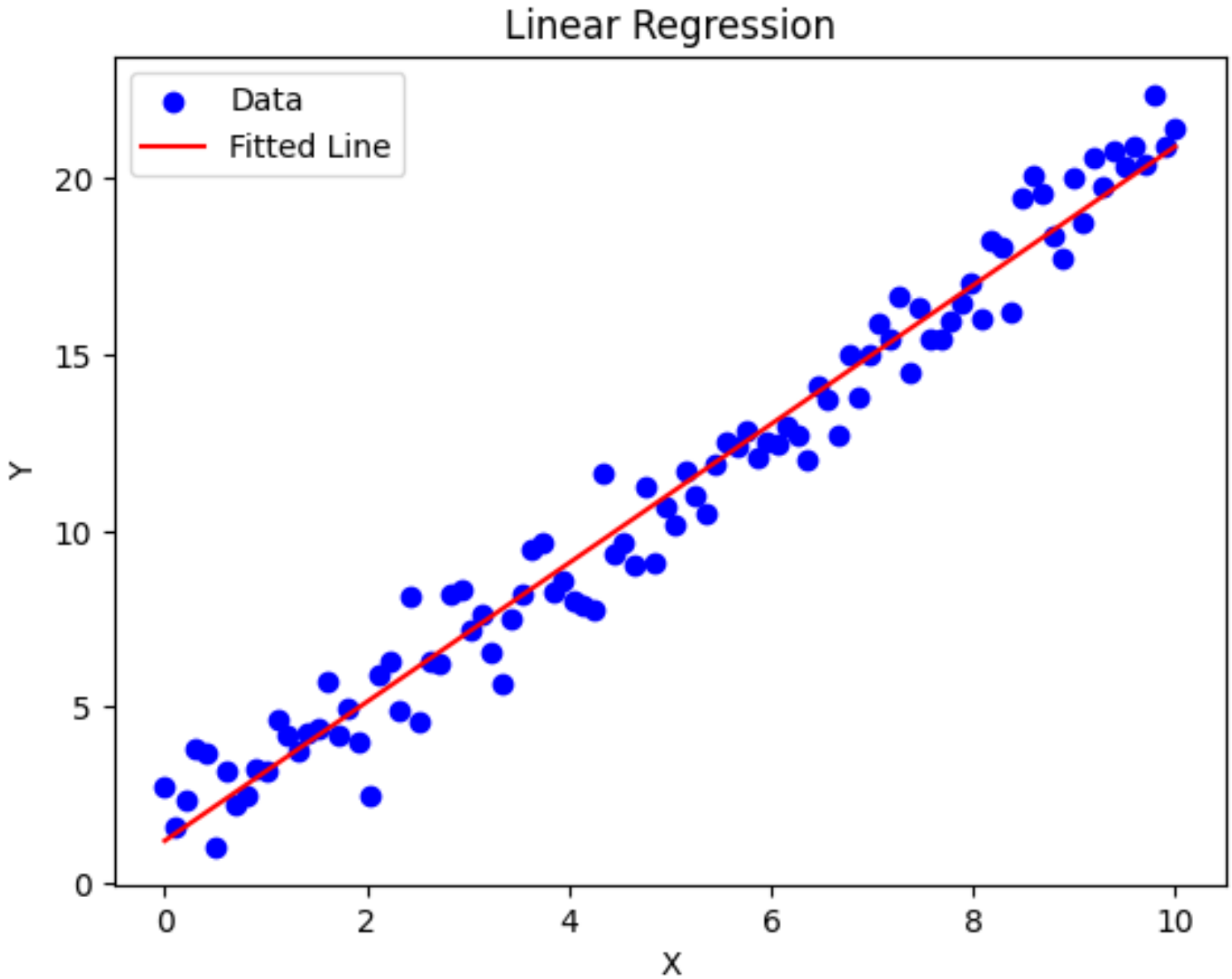
Advantages:

- Simple and fast
- Easy to interpret

Limitations:

- Works only for linear relationships
- Sensitive to outliers

Linear Regression Graph



Linear Regression Numerical Example

Given Data:

x	y
1	2
2	3
3	5
4	4
5	6

We compute: $\sum x, \sum y, \sum x^2, \sum xy$

Step 1: Required Calculations

x	y	x^2	xy
1	2	1	2
2	3	4	6
3	5	9	15
4	4	16	16
5	6	25	30

$$\sum x = 15, \quad \sum y = 20, \quad \sum x^2 = 55, \quad \sum xy = 69, \quad n = 5$$

Step 2: Formula for Slope

$$m = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

$$m = \frac{5(69) - (15)(20)}{5(55) - (15)^2}$$

$$m = \frac{345 - 300}{275 - 225} = \frac{45}{50} = 0.9$$

Step 3: Intercept Calculation

$$c = \frac{\sum y - m \sum x}{n}$$

$$c = \frac{20 - (0.9)(15)}{5}$$

$$c = \frac{20 - 13.5}{5} = \frac{6.5}{5} = 1.3$$

Final Regression Equation

$$y = mx + c$$

$$y = 0.9x + 1.3$$

Interpretation

- Slope $= 0.9 \rightarrow$ y increases as x increases
- Intercept $= 1.3 \rightarrow$ value of y when x $= 0$
- Best fit line: $y = 0.9x + 1.3$

Prediction Example

For $x = 6$:

$$y = 0.9(6) + 1.3 = 6.7$$

So predicted value = **6.7**

Linear Regression in Python (Code)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 3, 5, 4, 6])

model = LinearRegression()
model.fit(x, y)

y_pred = model.predict(x)

print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

Prediction Code

```
x_new = np.array([[6]])
y_new = model.predict(x_new)

print("Prediction for x=6:", y_new)
```

Visualization Code

```
plt.scatter(x, y, color='blue')
plt.plot(x, y_pred, color='red')
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression")
plt.show()
```

Key Points

- Linear Regression finds best-fit line
- Model learns slope and intercept
- Used for prediction and forecasting

Logistic Regression

Logistic Regression is a supervised learning algorithm used for classification problems.

It predicts the probability of an event belonging to a class (0 or 1).

Why Logistic Regression?

- Linear Regression cannot be used for classification
- Output of Linear Regression is not bounded between 0 and 1
- Logistic Regression fixes this using a sigmoid function

Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Converts input into probability
- Output lies between 0 and 1

Logistic Regression Model

$$y = \sigma(w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n)$$

- w_0 = bias
- w_i = weights
- x_i = input features

Decision Rule

- If probability $\geq 0.5 \rightarrow$ Class 1
- If probability $< 0.5 \rightarrow$ Class 0

Cost Function (Log Loss)

$$J = -\frac{1}{n} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- Measures error in classification
- Goal: minimize loss

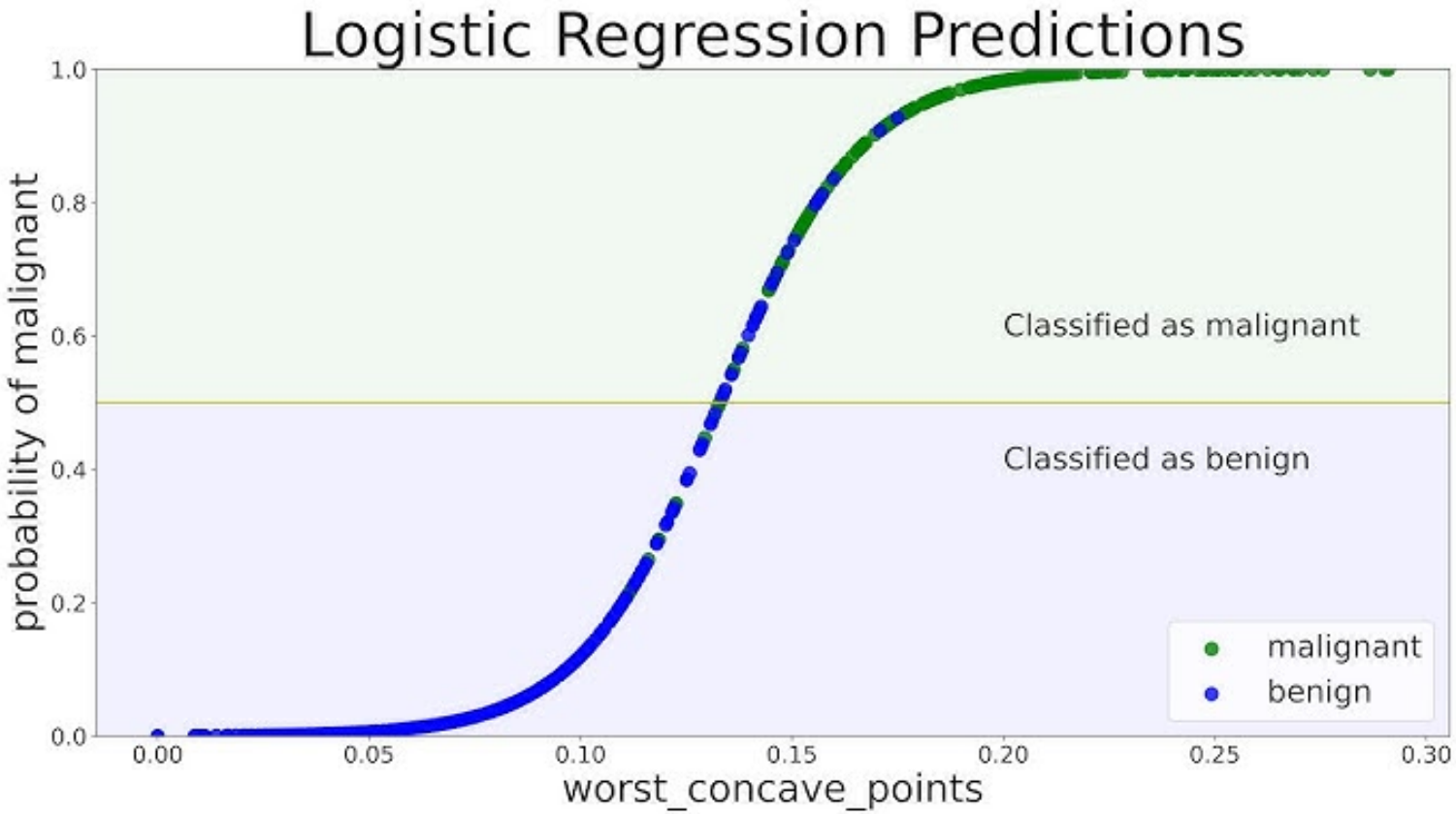
Applications

- Spam email detection
- Disease prediction
- Fraud detection
- Customer churn prediction

Conclusion

- Used for classification problems
- Outputs probability values
- Uses sigmoid function for mapping

Logistic Regression Graph



Logistic Regression Numerical Problem

Given:

$$z = 0.8x - 1.2$$

Find probability and class for $x = 2$.

Step 1: Compute z value

$$z = 0.8(2) - 1.2$$

$$z = 1.6 - 1.2 = 0.4$$

Step 2: Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$P = \frac{1}{1 + e^{-0.4}}$$

$$e^{-0.4} \approx 0.6703$$

$$P = \frac{1}{1 + 0.6703} = \frac{1}{1.6703} \approx 0.5987$$

Step 3: Decision Rule

- If $P \geq 0.5 \rightarrow$ Class 1
- If $P < 0.5 \rightarrow$ Class 0

Since:

$$0.5987 \geq 0.5$$

Predicted Class = 1

Final Result

- Probability $= 0.5987$
- Class $= 1$
- Interpretation: Positive class likely

Decision Tree

Decision Tree is a supervised machine learning algorithm used for both classification and regression.

It splits data into branches based on conditions to make decisions.

How Decision Tree Works

- Starts from Root Node
- Splits data based on feature conditions
- Creates branches and leaf nodes
- Leaf node gives final output

Structure of Decision Tree

- Root Node $\rightarrow$ Starting point
- Internal Nodes $\rightarrow$ Decision points
- Leaf Nodes $\rightarrow$ Final output/class

Splitting Criteria

Decision trees split data using:

- Gini Index
- Entropy (Information Gain)
- Chi-Square

Entropy

$$Entropy = -\sum p_i \log_2(p_i)$$

- Measures impurity in dataset
- Lower entropy $\rightarrow$ more pure data

Information Gain

$$IG = Entropy_{parent} - Entropy_{children}$$

- Higher information gain = better split

Advantages

- Easy to understand and visualize
- Works for classification and regression
- No need for feature scaling

Limitations

- Overfitting problem
- Sensitive to small data changes
- Can become very complex

Decision Tree Numerical Problem

We want to predict **Play Tennis (Yes/No)** using:

- Weather (Sunny, Overcast, Rainy)
- Temperature (Hot, Mild, Cool)
- Humidity (High, Normal)

Dataset

Weather	Temp	Humidity	Play
Sunny	Hot	High	No
Sunny	Hot	High	No
Overcast	Hot	High	Yes
Rainy	Mild	High	Yes
Rainy	Cool	Normal	Yes
Rainy	Cool	Normal	No
Overcast	Cool	Normal	Yes
Sunny	Mild	High	No

Step 1: Parent Entropy

$$Entropy(S) = 1$$

- Yes $= 4$
- No $= 4$

Step 2: Information Gain Formula

$$IG = Entropy(parent) - Entropy(split)$$

Goal: Select feature with highest Information Gain

Step 3: Weather Feature

- Sunny $\rightarrow$ Entropy $= 0$
- Overcast $\rightarrow$ Entropy $= 0$
- Rainy $\rightarrow$ Entropy $= 0.918$

$$Entropy_{weather} = 0.344$$

$$IG_{weather} = 1 - 0.344 = 0.656$$

Step 4: Temperature Feature

- Hot $\rightarrow$ Entropy 0.918
- Mild $\rightarrow$ Entropy $= 1$
- Cool $\rightarrow$ Entropy 0.918

$$IG_{temp} = 0.094$$

Step 5: Humidity Feature

- High $\rightarrow$ Entropy 0.971
- Normal $\rightarrow$ Entropy 0.918

$$IG_{humidity} = 0.049$$

Comparison of Features

Feature	Information Gain
Weather	0.656
Temperature	0.094
Humidity	0.049

Final Result

Root Node Selected: Weather

Because it has the highest Information Gain.

Final Tree

- Sunny $\rightarrow$ No
- Overcast $\rightarrow$ Yes
- Rainy $\rightarrow$ Further splitting required

Random Forest

Random Forest is an ensemble learning algorithm used for classification and regression.

It builds multiple decision trees and combines their outputs for final prediction.

Idea of Random Forest

- Single decision tree may overfit
- Random Forest uses multiple trees
- Final output is based on:
  - Majority voting (classification)
  - Average (regression)

How Random Forest Works

- Step 1: Random sampling (bootstrap sampling)
- Step 2: Build decision trees
- Step 3: Random feature selection at each split
- Step 4: Repeat for many trees
- Step 5: Combine outputs

Bootstrap Sampling

- Random samples are selected with replacement
- Each tree gets different training data
- Increases diversity among trees

Feature Randomness

- Only a subset of features is used at each split
- Reduces correlation between trees
- Improves model performance

Final Prediction

Classification:

$$\hat{y} = \text{majority vote of all trees}$$

Regression:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^{N} y_i$$

Advantages

- Reduces overfitting
- High accuracy
- Works well on large datasets

Disadvantages

- Slow compared to single tree
- Complex model
- Requires more memory

Applications

- Fraud detection
- Medical diagnosis
- Stock prediction
- Recommendation systems

Overview of Evaluation Metrics

To evaluate the performance of the classification model, several standard metrics are used to measure how well the model distinguishes between classes.

- Accuracy
- Precision
- Recall (Sensitivity)
- F1-Score
- Confusion Matrix
- ROC-AUC Score

These metrics together provide a balanced evaluation of model performance.

Accuracy and Precision

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Measures proportion of correctly classified instances.
- Limitation: can be misleading in imbalanced datasets.

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Measures how many predicted positives are actually correct.
- High precision indicates fewer false positives.

Recall and F1-Score

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Measures how many actual positives are correctly identified.
- High recall means fewer false negatives.

F1-Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Harmonic mean of precision and recall.
- Useful when balancing both metrics.

Confusion Matrix

	Predicted +	Predicted -
Actual +	*TP*	*FN*
Actual -	*FP*	*TN*

- Summarizes classification outcomes.
- Helps visualize types of prediction errors.

ROC-AUC Score and Conclusion

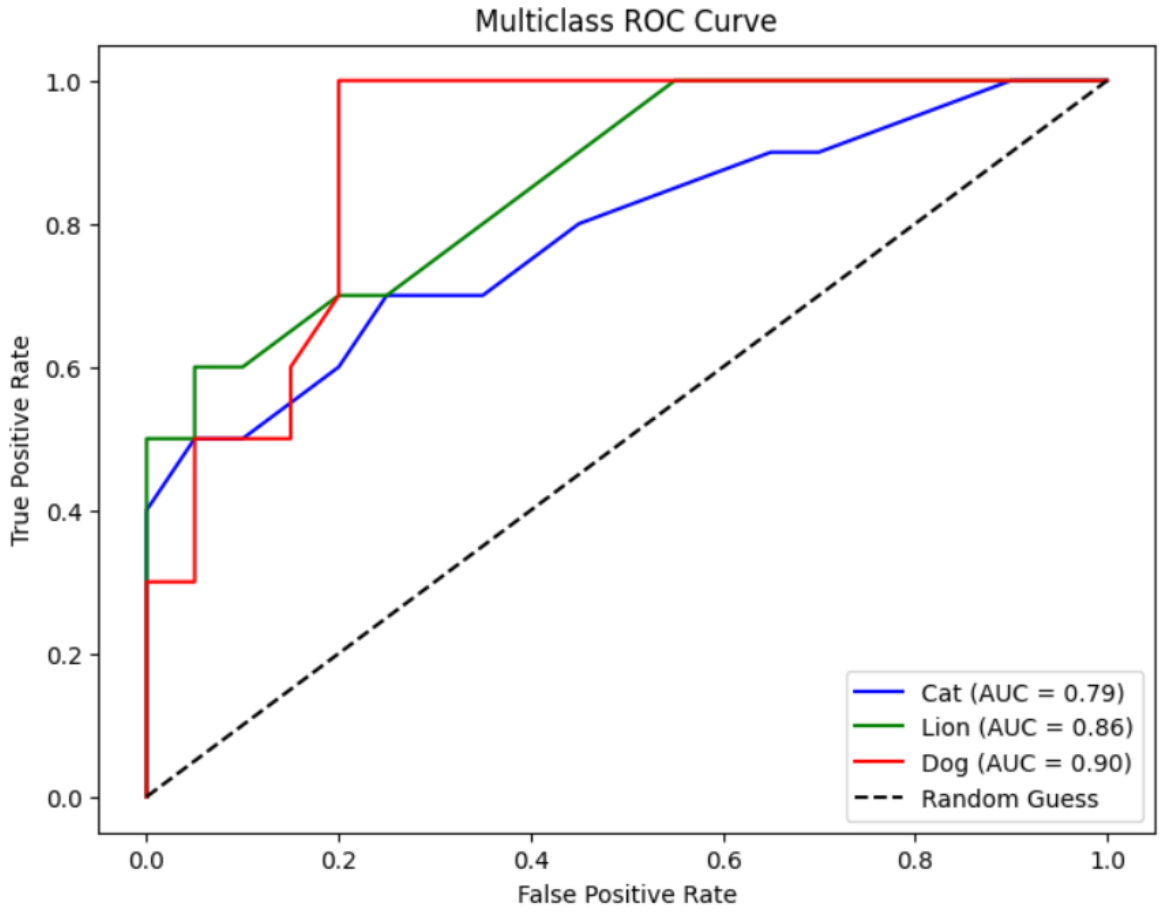
ROC-AUC Score

- ROC curve plots True Positive Rate vs False Positive Rate.
- AUC measures separability of classes.
- $0.5 \rightarrow$ no discrimination (random guess)
- $1.0 \rightarrow$ perfect classification

Conclusion

- These metrics collectively ensure a comprehensive evaluation of the classification model.
- They help understand both overall performance and error distribution.

ROC Curve


Multiclass ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Cat (AUC = 0.79)
Lion (AUC = 0.86)
Dog (AUC = 0.90)
Random Guess

Numerical Example – Classification Evaluation

Problem Statement: A model predicts whether a patient has a disease (Positive) or not (Negative) on 100 patients.

- Total patients $= 100$
- Actual Positive $= 50$, Actual Negative $= 50$
- TP $= 40$, FN $= 10$, TN $= 45$, FP $= 5$

Step 1: Accuracy and Precision

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{40 + 45}{100} = 0.85 = 85\%$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{40}{40 + 5} = \frac{40}{45} \approx 0.889 = 88.9\%$$

Step 2: Recall and F1-Score

Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{40}{40 + 10} = 0.80 = 80\%$$

F1-Score

$$F1 = 2 \cdot \frac{P \cdot R}{P + R} = 2 \cdot \frac{0.889 \cdot 0.80}{0.889 + 0.80} \approx 0.842 = 84.2\%$$

Confusion Matrix and Conclusion

Confusion Matrix

	Predicted +	Predicted -
Actual +	$TP = 40$	$FN = 10$
Actual -	$FP = 5$	$TN = 45$

Conclusion

- Accuracy = 85% (good overall performance)
- Precision = 88.9% (few false positives)
- Recall = 80% (some positives missed)
- F1-score = 84.2% (balanced performance)

Overfitting

Overfitting occurs when a model learns the training data too well, including noise and irrelevant patterns.

- Performs very well on training data but poorly on new/unseen data.
- Low training error but high testing error.
- Example: A model memorizing answers instead of learning concepts.

Key idea: Poor generalization.

Underfitting

Underfitting occurs when the model is too simple to learn the underlying patterns in the data.

- Performs poorly on both training and test data.
- High training error and high testing error.
- Example: A linear model trying to fit complex nonlinear data.

Key idea: Model fails to learn properly.

Bias–Variance Tradeoff

Bias

- Error due to overly simplified assumptions in the model.
- High bias $\rightarrow$ underfitting.

Variance

- Error due to model being too sensitive to training data.
- High variance $\rightarrow$ overfitting.

Tradeoff

- Goal is to balance bias and variance.
- Good model has:
  - Low bias
  - Low variance
  - Good generalization

Train, Validation, and Test Sets

Training Set

- Used to train the model (learn patterns).
- Largest portion of data.

Validation Set

- Used to tune hyperparameters.
- Helps prevent overfitting.

Test Set

- Used to evaluate final model performance.
- Not used during training or tuning.

Train–Test–Validation Split

A common data split is:

- Training set: 70%
- Validation set: 15%
- Test set: 15%

OR

- Training set: 80%
- Test set: 20% (if no validation set is used)

Unsupervised Learning

Unsupervised learning is a type of machine learning where the model is trained using **unlabeled data** (no target/output variable is given).

Key Idea

- The model discovers hidden patterns, structures, or relationships in data without supervision.

Characteristics

- No labeled outputs
- Only input data is given
- Finds hidden structure in data
- Used for pattern discovery and grouping

Applications

- Customer segmentation
- Market basket analysis
- Anomaly detection
- Image grouping

Types of Unsupervised Learning

Unsupervised learning is mainly divided into the following types:

(A) Clustering

Clustering is the process of grouping similar data points into clusters.

Key Idea

- Similar data points belong to the same cluster
- Dissimilar points belong to different clusters

Common Algorithms

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN (Density-Based Clustering)

Applications

- Customer segmentation
- Social network grouping
- Image segmentation

(B) Association Rule Learning

This technique finds relationships between variables in large datasets.

Key Idea

- Identifies rules like: "If A happens, B is likely to happen"

Example

- If a customer buys bread $\rightarrow$ they may also buy butter

Common Algorithms

- Apriori Algorithm
- FP-Growth Algorithm

Applications

- Market basket analysis
- Recommendation systems
- Retail analysis

(C) Dimensionality Reduction

This technique reduces the number of input variables while preserving important information.

Key Idea

- Converts high-dimensional data into lower dimensions
- Removes noise and redundancy

Common Methods

- PCA (Principal Component Analysis)
- t-SNE
- LDA (sometimes supervised variant)

Applications

- Data visualization
- Feature reduction
- Noise reduction
- Image compression

Summary

- Unsupervised learning works on unlabeled data.
- It is mainly divided into:
  - Clustering (grouping data)
  - Association Rule Learning (finding relationships)
  - Dimensionality Reduction (reducing features)

K-Means Clustering

K-Means is a **partition-based clustering algorithm** used in unsupervised learning that divides data into K clusters.

Key Idea:

- Data points in the same cluster are similar
- Data points in different clusters are dissimilar
- Each cluster is represented by a centroid (mean point)

Steps of K-Means Algorithm

- Step 1: Choose number of clusters (K)
- Step 2: Initialize K centroids randomly
- Step 3: Assign points to nearest centroid
- Step 4: Update centroid (mean of points)
- Step 5: Repeat until convergence

Key Concepts

Distance Measure:

- Usually Euclidean distance is used
- Determines nearest centroid

Objective:

- Minimize within-cluster variation
- Also called WCSS (Within Cluster Sum of Squares)

Advantages and Limitations

Advantages

- Simple and easy to implement
- Works well for large datasets
- Fast convergence

Limitations

- K must be chosen in advance
- Sensitive to initial centroids
- Poor performance for non-spherical clusters

Numerical Example – Initial Step

Given points:
A(1,1), B(1.5,2), C(3,4), D(5,7), E(3.5,5), F(4.5,5)

Let K = 2

Initial Centroids:

- C1 = A(1,1)
- C2 = D(5,7)

Cluster Formation

After computing distances:

Cluster 1:

- A(1,1), B(1.5,2)

Cluster 2:

- C(3,4), D(5,7), E(3.5,5), F(4.5,5)

Updated Centroids

New Centroid of Cluster 1

$$C1 = (1.25, 1.5)$$

New Centroid of Cluster 2

$$C2 = (4, 5.25)$$

Clusters remain unchanged after recomputation $\rightarrow$ convergence achieved.

Hierarchical Clustering – Theory

Hierarchical clustering is an **unsupervised learning algorithm** that builds a hierarchy of clusters.

Key Idea

- Data is grouped based on similarity (distance)
- It creates a tree-like structure called a dendrogram
- Final clusters are obtained by cutting the dendrogram

It does not require K in advance

Types of Hierarchical Clustering

1. **Agglomerative (Bottom-Up)**
   - Each data point starts as its own cluster
   - Clusters are merged step by step
   - Most commonly used method
2. **Divisive (Top-Down)**
   - All data points start in one cluster
   - Cluster is split step by step
   - Less commonly used

Distance and Linkage Methods

Distance Measures

- Euclidean distance
- Manhattan distance

Linkage Methods

- Single linkage $\rightarrow$ minimum distance
- Complete linkage $\rightarrow$ maximum distance
- Average linkage $\rightarrow$ average distance

Applications

- Customer segmentation
- Gene classification
- Image grouping
- Document clustering

Numerical Example – Data Points

Given data points:

- A(1,1)
- B(2,1)
- C(4,3)
- D(5,4)

Step 1: Distance Calculation

Euclidean distances:

- $AB = 1$
- $AC = \sqrt{13} \approx 3.61$
- $AD = 5$
- $BC = \sqrt{8} \approx 2.83$
- $BD = \sqrt{18} \approx 4.24$
- $CD = \sqrt{2} \approx 1.41$

Step 2: First Merge

Smallest distance = **AB = 1**

So,

- Cluster 1 = (A, B)

Step 3 and 4: Update and Recompute

Now clusters:

- (AB), C, D

Single Linkage Distances:

- (AB, C) = min(AC, BC) = 2.83
- (AB, D) = min(AD, BD) = 4.24
- (C, D) = 1.41

Step 5: Second Merge

Smallest distance = **CD = 1.41**

So,

- Cluster 2 = (C, D)

Step 6: Final Merge

Now clusters:

- (AB)
- (CD)

Distance:

- min(AC, AD, BC, BD) = 2.83

Final cluster:

- (ABCD)

Final Hierarchy

- A + B $\rightarrow$ Cluster at distance 1
- C + D $\rightarrow$ Cluster at distance 1.41
- (AB) + (CD) $\rightarrow$ Final cluster at distance 2.83

Dimensionality Reduction – Definition

Dimensionality reduction is a technique in machine learning used to reduce the number of input variables (features) in a dataset while preserving important information.

It transforms high-dimensional data into a lower-dimensional form.

Key Idea

- Real-world datasets often have many features.
- Not all features are useful.
- Removes redundant, irrelevant, or noisy features.

Why Dimensionality Reduction is Needed

- Reduces computational cost
- Improves model performance
- Reduces overfitting
- Helps in data visualization (2D/3D)
- Removes noise and redundancy

Types of Dimensionality Reduction

(A) Feature Selection

- Selects important features from dataset
- No transformation applied
- Methods:
  - Filter methods (correlation, chi-square)
  - Wrapper methods (forward/backward selection)
  - Embedded methods (Lasso regression)

(B) Feature Extraction

- Creates new features from original data
- Transforms data into lower-dimensional space
- Methods:
  - PCA (Principal Component Analysis)
  - t-SNE
  - LDA

PCA (Principal Component Analysis)

- Most commonly used technique
- Converts correlated variables into uncorrelated principal components
- First component captures maximum variance

Applications

- Data visualization (2D/3D plots)
- Image compression
- Face recognition
- Noise reduction
- Feature reduction in machine learning models

Advantages and Limitations

Advantages

- Reduces computation time
- Improves model accuracy
- Removes multicollinearity
- Helps visualize complex data

Limitations

- Some information may be lost
- Hard to interpret transformed features
- Not suitable for all datasets

Principal Component Analysis (PCA) – Definition

Principal Component Analysis (PCA) is a feature extraction technique used in dimensionality reduction.

It transforms correlated variables into a new set of uncorrelated variables called **principal components**.

Key Idea:

- Reduce dimensionality while preserving maximum variance
- First principal component captures maximum information

Why PCA is Used

- Reduces number of features
- Removes correlation between variables
- Improves visualization (2D/3D)
- Reduces overfitting
- Improves computational efficiency

Steps in PCA

- Step 1: Standardize the data
- Step 2: Compute covariance matrix
- Step 3: Compute eigenvalues and eigenvectors
- Step 4: Sort eigenvalues in descending order
- Step 5: Select top principal components
- Step 6: Transform data into new space

Numerical Example – Data

Given data points:

- (2,1)
- (3,5)
- (4,3)
- (5,6)

Step 1: Mean Calculation

$$\bar{X} = \frac{2+3+4+5}{4} = 3.5$$

$$\bar{Y} = \frac{1+5+3+6}{4} = 3.75$$

Step 2: Centered Data

- (2-3.5, 1-3.75) = (-1.5, -2.75)
- (3-3.5, 5-3.75) = (-0.5, 1.25)
- (4-3.5, 3-3.75) = (0.5, -0.75)
- (5-3.5, 6-3.75) = (1.5, 2.25)

Step 3: Covariance Matrix

$$\begin{bmatrix} 1.67 & 2.08 \\ 2.08 & 3.58 \end{bmatrix}$$

Step 4: Eigenvalues and Eigenvectors

Eigenvalues:

- $\lambda_1 \approx 5.1$
- $\lambda_2 \approx 0.15$

Eigenvectors:

- PC1 $\approx (0.62, 0.78)$
- PC2 $\approx (-0.78, 0.62)$

Final Result

- Largest eigenvalue: $\lambda_1 = 5.1$
- Select PC1 only
- Data reduced from 2D $\rightarrow$ 1D

Conclusion: PCA preserves maximum variance while reducing dimensionality.

Evaluation Techniques for Clustering

Clustering is an unsupervised learning method, so there are no true labels available.

Therefore, special evaluation techniques are used to measure the quality of clustering results.

Clustering evaluation is divided into:

- Internal Evaluation
- External Evaluation
- Relative Evaluation

Internal Evaluation Methods

Internal evaluation uses only the dataset (no true labels required).

1. **Silhouette Score**
   - Measures similarity within cluster vs other clusters
   - Range: -1 to $+1$
   - $+1 \rightarrow$ good clustering, $0 \rightarrow$ overlap, -1 $\rightarrow$ wrong clustering

2. **Davies-Bouldin Index (DBI)**
   - Measures similarity between clusters
   - Lower value is better

More Internal Measures

3. Dunn Index

- Ratio of minimum inter-cluster distance to maximum intra-cluster distance
- Higher value indicates better clustering

External Evaluation Methods

External evaluation uses true labels to evaluate clustering quality.

1. **Purity**
   - Measures how much a cluster contains a single class
   - Higher purity is better

2. **Rand Index (RI)**
   - Measures similarity between predicted and actual clustering
   - Considers TP, TN, FP, FN

3. **Adjusted Rand Index (ARI)**
   - Improved version of RI
   - Adjusts for random chance
   - 1 = perfect clustering, 0 = random

Relative Evaluation Methods

Used to compare different clustering models

- Helps select best number of clusters (K)
- Common methods:
  - Elbow Method
  - Silhouette Analysis

Elbow Method

- Plot WCSS vs number of clusters (K)
- Choose K at the "elbow point"
- After elbow, improvement becomes small

Key Idea:

- Best K is where curve bends sharply

Summary

- Internal evaluation: no labels (Silhouette, DBI, Dunn)
- External evaluation: uses true labels (Purity, RI, ARI)
- Relative evaluation: compares models (Elbow method)

Clustering evaluation helps in selecting the best clustering model and optimal number of clusters.

Thank You